30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 7

Dependency Injection — Principles, Hilt, Dagger Internals, Koin & KMP



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 7

Why DI is Day 7

DI questions separate engineers who add annotations from engineers who understand **object graphs**. What interviewers expect in 2026:

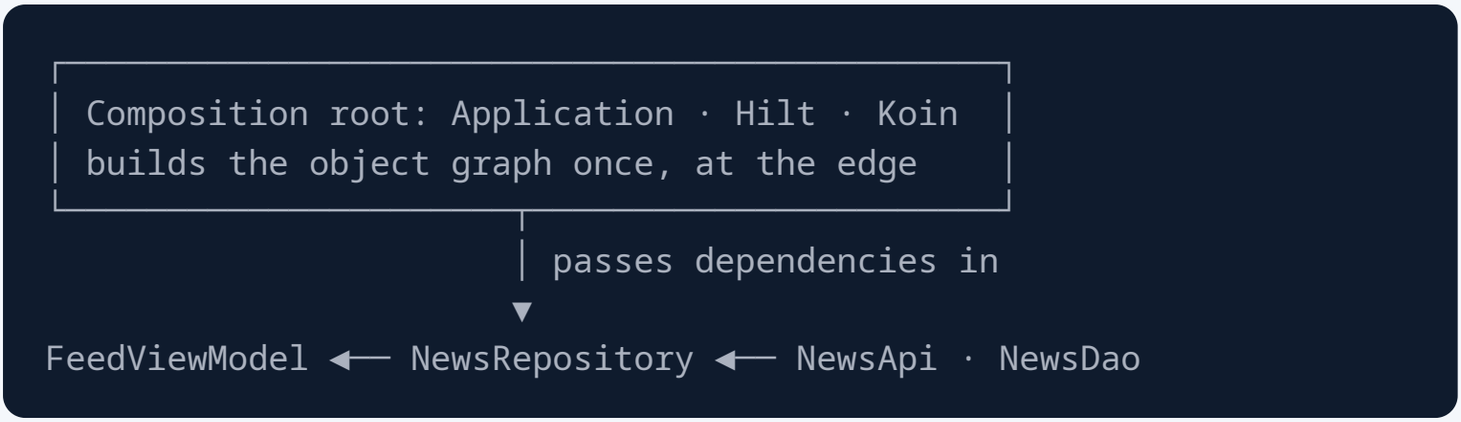
- DI as a **principle** first: constructor injection, dependency inversion, testability
- **Hilt** as the Android default: components, scopes, qualifiers, assisted injection, testing
- What **Dagger generates** — and why its errors show up at compile time
- **Koin** and **manual DI** as serious options, especially in Kotlin Multiplatform
- kapt is in maintenance mode — Dagger and Hilt run on **KSP**

Level	Questions
Junior	What DI is · Constructor vs field injection · Hilt setup
Mid	Components & scopes · Binds, Provides & qualifiers · Assisted injection · Testing
Senior	What Dagger generates · Multibindings & modules · Lazy, Provider & cycles
Architect	Hilt vs Koin vs manual · DI in KMP · DI at scale



Esmaeeil Moradi
@esmaeeil-moradi700

Mental model: one composition root



- Classes **declare** what they need — in the constructor — and never create it
- One place, the **composition root**, decides which implementation each class gets
- Hilt, Koin or a hand-written container only **write that place for you**

DI IS A PATTERN BEFORE IT'S A LIBRARY

If you can explain your graph without naming a framework, you understand DI.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 7

Junior

What DI is · Constructor vs field injection · Hilt setup



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. What is dependency injection?

```
// Without DI: hidden, hard-wired dependencies
class NewsRepository {
    private val api = RetrofitNewsApi()
    private val dao = AppDatabase.instance.newsDao()
}

// With DI: dependencies are visible and replaceable
class NewsRepository(
    private val api: NewsApi,
    private val dao: NewsDao,
)

// A unit test just passes fakes
val repo = NewsRepository(FakeNewsApi(), FakeNewsDao())
```

A class **receives** its dependencies instead of creating or looking them up. That's all DI is — the library is optional.



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. Why it matters — and what it isn't

- **Dependency inversion:** depend on `NewsApi` (an interface), not `RetrofitNewsApi` — swap implementations without touching the class
- **Testability:** pass fakes through the constructor — no framework in unit tests
- **An explicit graph:** the constructor lists everything the class needs

	Dependency injection	Service locator
Dependencies	Visible in the constructor	Hidden inside the class (`get()`)
Missing dependency	Found when wiring	Crash when first used
Tests	Pass fakes directly	Configure a global registry

INTERVIEW TRAP

"DI means Dagger or Hilt." — DI means passing dependencies in. A framework only automates the wiring; a small app can do it by hand (Q11).



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Constructor, field or method injection?

Type	Use when
Constructor	Your own classes — the default
Field (`@Inject lateinit var`)	Android creates the object: Activity, Fragment, Service, BroadcastReceiver
Method (`@Inject fun`)	Rare: something must run after construction

```
@AndroidEntryPoint
class PlayerService : Service() {
    @Inject lateinit var player: AudioPlayer

    override fun onCreate() {
        super.onCreate() // fields are injected here
        player.prepare()
    }
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Timing and traps

Entry point	Injected fields are ready after
Activity	`super.onCreate()`
Fragment	`super.onAttach()`
Service	`super.onCreate()`
BroadcastReceiver	`super.onReceive()`

INTERVIEW TRAP

Field injection in your own classes (ViewModels, repositories). Dependencies are hidden, objects can exist half-initialized, and every test needs a framework. Use constructor injection.

INTERVIEW TRAP

Passing a repository down the composable tree. Composables aren't injected: the **route** obtains the ViewModel (`hiltViewModel()`), everything below receives state and lambdas (Day 3).

FOLLOW-UP

"Can a Fragment use constructor injection?" — Yes, with a `FragmentFactory` (Day 2). Hilt simply uses field injection for Fragments.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Hilt setup in a modern project

```
@HiltAndroidApp
class App : Application()

@AndroidEntryPoint
class MainActivity : ComponentActivity()

@HiltViewModel
class FeedViewModel @Inject constructor(
    private val repo: NewsRepository,
    private val handle: SavedStateHandle, // provided by Hilt
) : ViewModel()

@Composable
fun FeedRoute(viewModel: FeedViewModel = hiltViewModel()) {
    /* collect state, render the screen */
}
```

Gradle: the `com.google.dagger.hilt.android` plugin, `hilt-android`, and `hilt-compiler` added with **`ksp(…)`**, not `kapt(…)`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. What each annotation does

Annotation	What it does
`@HiltAndroidApp`	Generates the root `SingletonComponent`; collects every `@InstallIn` module
`@AndroidEntryPoint`	Gives an Activity, Fragment, View, Service or BroadcastReceiver its component and injects fields
`@HiltViewModel`	Registers the ViewModel; `hiltViewModel()` / `by viewModels()` create it
`@Inject constructor`	How to build **your** class
`@Module` + `@InstallIn`	How to build types you **don't own** (interfaces, Retrofit, Room)
`@EntryPoint`	Reach the graph where Hilt can't inject (e.g. a `ContentProvider`)

INTERVIEW TRAP

An `@AndroidEntryPoint` Fragment inside an Activity that isn't annotated → crash at runtime. Hilt requires the host activity to be an entry point too.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 7



Mid-Level

Components & scopes · Binds, Provides & qualifiers · Assisted injection · Testing



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Hilt components and scopes

```
SingletonComponent              @Singleton
├─ ActivityRetainedComponent    @ActivityRetainedScoped
│  ├─ ViewModelComponent        @ViewModelScoped
│  └─ ActivityComponent         @ActivityScoped
│     ├─ FragmentComponent      @FragmentScoped
│     └─ ViewComponent          @ViewScoped
└─ ServiceComponent             @ServiceScoped
```

- A child component sees every binding of its parents — never the reverse. (`ViewWithFragmentComponent` sits under `FragmentComponent`.)
- A scoped binding = **one instance per component instance**. Unscoped = a new instance for every injection.
- `ViewModelComponent` provides `SavedStateHandle` automatically.



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Choosing a scope

Scope	One instance per…	Typical use
`@Singleton`	App process	`OkHttpClient`, the database, shared caches
`@ActivityRetainedScoped`	Activity, surviving rotation	State shared by an activity's ViewModels
`@ViewModelScoped`	ViewModel	One object shared by all of a ViewModel's dependencies
`@ActivityScoped` / `@FragmentScoped`	Activity / Fragment instance	Rare: helpers tied to that UI instance

INTERVIEW TRAP

"ViewModels live in `ActivityRetainedComponent`" and *"a Fragment's ViewModel lives in `FragmentComponent`"* (v1). — Every `@HiltViewModel` gets its **own** `ViewModelComponent`; its lifetime comes from its `ViewModelStoreOwner` (Day 6).

INTERVIEW TRAP

"Scope everything for performance." — Scoping costs generated code and a double-checked lock (`DoubleCheck`), and the object stays in memory as long as its component. Google's guidance: scope sparingly.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. `@Binds`, `@Provides` and qualifiers

```
@Module
@InstallIn(SingletonComponent::class)
abstract class DataModule {
    @Binds
    abstract fun newsRepo(
        impl: OfflineFirstNewsRepository,
    ): NewsRepository
}

@Module
@InstallIn(SingletonComponent::class)
object DispatchersModule {
    @Provides @IoDispatcher
    fun io(): CoroutineDispatcher = Dispatchers.IO
}

@Qualifier @Retention(AnnotationRetention.BINARY)
annotation class IoDispatcher
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Which one, and why

	`@Binds`	`@Provides`
Declared as	Abstract function, one parameter	Function with a body
Use for	Interface → your implementation	Types you don't own: Retrofit, Room, OkHttp, dispatchers
Generated code	Less: Dagger calls the implementation's factory directly	A factory per method

INTERVIEW TRAP

Two bindings of the same type without a qualifier → compile error `[Dagger/DuplicateBindings]`. Prefer custom `@Qualifier` annotations over `@Named("io")`: type-checked, refactor-safe and self-documenting.

FOLLOW-UP

"Different implementations for debug and release?" — Decide at **build time**: a module per source set (`src/debug` , `src/release`). Only real runtime decisions (feature flags) need runtime selection (Q13).



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Assisted injection: runtime parameters

```
class ChatSession @AssistedInject constructor(
    private val api: ChatApi,
    @Assisted private val roomId: String,
) {
    @AssistedFactory
    interface Factory {
        fun create(roomId: String): ChatSession
    }
}

// Inject the factory anywhere
class ChatManager @Inject constructor(
    private val sessions: ChatSession.Factory,
) {
    fun open(roomId: String) = sessions.create(roomId)
}
```

Dagger supplies `api`; the caller supplies `roomId`. No hand-written factory class.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Assisted ViewModels

```
@HiltViewModel(assistedFactory = ItemViewModel.Factory::class)
class ItemViewModel @AssistedInject constructor(
    @Assisted val item: ItemPreview,
    private val repo: ItemRepository,
) : ViewModel() {
    @AssistedFactory
    interface Factory {
        fun create(item: ItemPreview): ItemViewModel
    }
}

// Compose
val vm = hiltViewModel<ItemViewModel, ItemViewModel.Factory> {
    it.create(preview)
}
```

INTERVIEW TRAP

Assisted injection for navigation arguments. They're already in `SavedStateHandle` and survive process death. Use it for values that aren't in saved state.

INTERVIEW TRAP

Assisted types **can't be scoped** — every `create()` is a new instance. Two `@Assisted` parameters of the same type need identifiers: `@Assisted("from")`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Testing with Hilt

```
@HiltAndroidTest
@UninstallModules(PaymentModule::class)
class CheckoutFlowTest {
    @get:Rule(order = 0) val hiltRule = HiltAndroidRule(this)

    @get:Rule(order = 1)
    val composeRule = createAndroidComposeRule<MainActivity>()

    @BindValue @JvmField
    val gateway: PaymentGateway = FakePaymentGateway()

    @Test fun paysWithTheFakeGateway() {
        composeRule.onNodeWithText("Pay").performClick()
        composeRule.onNodeWithText("Paid").assertIsDisplayed()
    }
}
```

`@BindValue` replaces one binding for this test class; the original module must be uninstalled, or you get a duplicate binding.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Replacing modules everywhere

```
@Module
@TestInstallIn(
    components = [SingletonComponent::class],
    replaces = [PaymentModule::class],
)
abstract class FakePaymentModule {
    @Binds
    abstract fun gateway(
        fake: FakePaymentGateway,
    ): PaymentGateway
}
```

- Instrumented tests: a custom `AndroidJUnitRunner` whose `newApplication()` returns `HiltTestApplication`. Robolectric: `@Config(application = HiltTestApplication::class)`.

INTERVIEW TRAP

v1 set `testInstrumentationRunner` to `"androidx.hilt.work.testing.HiltTestApplication_Application"` "for unit tests". No such class — and that setting only affects **instrumented** tests.

INTERVIEW TRAP

"ViewModel unit tests need Hilt." — No. Constructor injection means `FeedViewModel(FakeNewsRepository())`. Hilt tests are for integration and UI tests.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 7

Senior

What Dagger generates · Multibindings & modules · Lazy, Provider & cycles



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. What Dagger generates

You write	Dagger generates (Java)
`@Inject constructor`	`UserRepo_Factory`
`@Provides fun api()`	`NetworkModule_ApiFactory`
`@Component / Hilt root`	the component class
`@Singleton binding`	DoubleCheck memoization

```
// Simplified shape of the generated code
class UserRepo_Factory(
    private val api: Provider<Api>,
    private val dao: Provider<UserDao>,
) : Factory<UserRepo> {
    override fun get() = UserRepo(api.get(), dao.get())
}

// Inside the component, for a @Singleton binding:
val userRepo: Provider<UserRepo> = DoubleCheck.provider(
    UserRepo_Factory(apiProvider, daoProvider),
)
```

No reflection at runtime — just constructor calls. The whole graph is validated **at compile time**.



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. What Hilt adds on top

- `@HiltAndroidApp` generates a `Hilt_App` base class; the Hilt Gradle plugin rewrites your superclass in bytecode, so you still write `: Application()`.
- When `:app` compiles, Hilt **aggregates every `@InstallIn` module** in its dependency graph and generates the whole component hierarchy.
- Graph problems are compile errors: `[Dagger/MissingBinding]`, `[Dagger/DependencyCycle]`, `[Dagger/DuplicateBindings]`.

INTERVIEW TRAP

"Dagger is slow because it uses reflection." — Dagger 2 generates plain code. Its cost is **build time** (annotation processing), not runtime.

INTERVIEW TRAP

"`@Singleton` objects are created at app start." — Dagger creates them lazily, on first request. But everything injected into `Application` fields is created in `Application.onCreate()` — startup cost (Day 1).



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Multibindings: modules contribute, the app collects

```
interface AppInitializer { fun init(app: Application) }

// :feature:analytics:impl
@Module @InstallIn(SingletonComponent::class)
abstract class AnalyticsInitModule {
    @Binds @IntoSet
    abstract fun bind(
        impl: AnalyticsInitializer,
    ): AppInitializer
}

@HiltAndroidApp
class App : Application() {
    @Inject lateinit var initializers:
        Set<@JvmSuppressWildcards AppInitializer>

    override fun onCreate() {
        super.onCreate()
        initializers.forEach { it.init(this) }
    }
}
```

`@IntoSet`, `@IntoMap` (with `@StringKey` / `@ClassKey`) and `@Multibinds` (a collection that may be empty). Day 2 used the same idea to contribute navigation graphs.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. DI across Gradle modules

- `@HiltAndroidApp` lives only in `:app`. Hilt builds **one** component hierarchy from every `@InstallIn` module `:app` depends on.
- Interfaces in `:feature:x:api` (or `:core:domain`); implementations and their `@Binds` in `:impl`. Features depend only on `:api`.
- Make implementations `internal` — Gradle visibility, not Hilt, is your encapsulation.

INTERVIEW TRAP

"Each module manages its own walled-garden graph." (v1) — With Hilt there is one graph: a binding in `SingletonComponent` is visible to every module that can see its type.

INTERVIEW TRAP

"`@AndroidEntryPoint` works inside dynamic feature modules; Hilt adds their modules when downloaded." (v1) — No. The base app can't know a dynamic feature at compile time, so Hilt can't process it. Use plain Dagger in the feature, with component dependencies on an `@EntryPoint` from the app.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. `Lazy`, `Provider` and dependency cycles

```
class ReportExporter @Inject constructor(
    // dagger.Lazy: created on first get(), then cached
    private val pdf: Lazy<PdfRenderer>,
    // new instance per get() — unless the binding is scoped
    private val upload: Provider<UploadRequest>,
) {
    fun export(report: Report) {
        val file = pdf.get().render(report)
        upload.get().send(file)
    }
}

class A @Inject constructor(private val b: B)
class B @Inject constructor(private val a: A)
// compile error: [Dagger/DependencyCycle]
```

`Lazy` keeps heavy, rarely used objects off the startup path.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. What people get wrong

INTERVIEW TRAP

"`Provider.get()` always returns a new instance." (v1) — Only for unscoped bindings. For a scoped binding every `get()` returns the same instance.

INTERVIEW TRAP

"A cycle is a compile error or a `StackOverflowError` at runtime." (v1) — Dagger reports every cycle at compile time. It compiles only if one edge is `Lazy` or `Provider` — and still overflows if you call `get()` inside a constructor.

INTERVIEW TRAP

Importing `kotlin.Lazy` instead of `dagger.Lazy` → `[Dagger/MissingBinding]`. Kotlin's `Lazy` isn't a Dagger type.

FIX CYCLES BY DESIGN

Extract the shared logic into a third class, or invert one edge with an interface, a callback or a `Flow`. `Lazy` and `Provider` only hide the smell.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 7

→

Architect

Hilt vs Koin vs manual · DI in KMP · DI at scale



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Hilt, Koin or manual DI?

	Hilt / Dagger	Koin	Manual
Graph checked	Compile time	Runtime — or a `verify()` test	By the compiler: plain code
Build cost	Annotation processing (KSP)	None	None
Runtime	Generated code, no reflection	Registry lookup when resolving	Direct calls
Multiplatform	No — JVM / Android	Yes	Yes
Sweet spot	Large Android apps, many teams	KMP, small-to-mid apps	Small apps, libraries, SDKs

Compile-time options for KMP also exist: **kotlin-inject** (KSP) and **Metro** (a compiler plugin).

INTERVIEW TRAP

"Koin is just a service locator." — Module definitions with constructor parameters are DI. Calling `get()` or `by inject()` **inside** business classes is service location: hidden dependencies, runtime failures.



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. The same graph in Koin and by hand

```
// Koin
val appModule = module {
    singleOf(::OfflineFirstNewsRepository) {
        bind<NewsRepository>()
    }
    viewModelOf(::FeedViewModel)
}

// Manual DI: a container is just a class
class AppContainer(context: Context) {
    private val db by lazy { AppDatabase.build(context) }
    private val api by lazy { NewsApi(HttpClient()) }
    val newsRepository: NewsRepository by lazy {
        OfflineFirstNewsRepository(db.newsDao(), api)
    }
}
```

- Koin: add a `verify()` unit test (koin-test) so a missing definition fails in CI, not on a user's phone.
- Manual DI plugs into ViewModels through `viewModelFactory` + `APPLICATION_KEY` (Day 6).
- Writing an SDK? Don't force a DI framework on your users — expose constructors or a builder.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. DI in Kotlin Multiplatform

```
// commonMain
expect val platformModule: Module

val sharedModule = module {
    singleOf(::OfflineFirstNewsRepository) {
        bind<NewsRepository>()
    }
    viewModelOf(::FeedViewModel)
}

fun initKoin(config: KoinAppDeclaration = {}) = startKoin {
    config()
    modules(sharedModule, platformModule)
}

// androidMain
actual val platformModule = module {
    single<DbFactory> { AndroidDbFactory(androidContext()) }
}
```

Android calls `initKoin { androidContext(this@App) }`. `iosMain` binds `IosDbFactory()`; Swift sees the function as `doInitKoin()`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Rules for shared code

- Interfaces live in `commonMain`; platform implementations are **injected**. Keep `expect` / `actual` for the small platform factory at the edge.
- Options: Koin (runtime, the most common), kotlin-inject or Metro (compile-time), or manual containers.
- Hilt can stay in the Android app module and `@Provides` the shared classes — it just can't live in `commonMain`.

INTERVIEW TRAP

"Use Hilt in shared code." — Dagger and Hilt are JVM/Android only; they can't run on iOS.

INTERVIEW TRAP

An `expect class` for every platform dependency. It hard-wires one implementation per platform and leaves no room for fakes in common tests. Prefer interfaces + DI.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Switching implementations at runtime

```
@Module @InstallIn(SingletonComponent::class)
abstract class CheckoutModule {
    @Binds @IntoMap @StringKey("classic")
    abstract fun classic(impl: ClassicCheckout): Checkout

    @Binds @IntoMap @StringKey("express")
    abstract fun express(impl: ExpressCheckout): Checkout
}

class CheckoutSelector @Inject constructor(
    private val variants:
        Map<String, @JvmSuppressWildcards Provider<Checkout>>,
    private val flags: FeatureFlags,
) {
    fun current(): Checkout =
        variants.getValue(flags.checkoutVariant()).get()
}
```

The graph stays static; only the **choice** happens at runtime. With `Provider`, only the selected implementation is created.

INTERVIEW TRAP

v1's "hot-swapping DI system" (a custom switcher reached via entry points) is overkill. A Dagger graph is fixed at compile time — bind the candidates, then select.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Keeping a large graph fast and healthy

Concern	Practice
Build time	Dagger/Hilt on KSP; `@Binds` over `@Provides`; small, stable modules
Startup	Nothing heavy in `Application` fields; `Lazy` for rarely used heavy objects
Memory	Scope only shared state or expensive objects
Correctness	Compile-time graph errors; `dagger.fullBindingGraphValidation` also checks unused bindings; Koin `verify()` in CI
Boundaries	`api` / `impl` modules, `internal` implementations

SENIOR ANSWER IN ONE LINE

"Constructors declare what they need, one composition root decides what they get — and the framework, Hilt, Koin or none, is a trade-off I can defend."



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 7 · ARCHITECT

Day 7 — cheat sheet

- DI = receive dependencies; don't create or look them up. A framework only writes the wiring.
- Constructor injection by default; field injection only where Android creates the object.
- Hilt: `@HiltAndroidApp`, `@AndroidEntryPoint`, `@HiltViewModel`, `@InstallIn` modules — compiled with KSP.
- Each ViewModel gets its own `ViewModelComponent`. Scope only shared state or expensive objects.
- `@Binds` for your interfaces, `@Provides` for types you don't own, custom qualifiers over `@Named`.
- Assisted injection for runtime values; navigation args already live in `SavedStateHandle`.
- Unit tests call constructors with fakes; `@TestInstallIn` / `@BindValue` for integration and UI tests.
- Dagger generates plain code and checks the graph at compile time; `@Singleton` is created lazily.
- `Provider` is "new every time" only for unscoped bindings. Fix cycles by design, not with `Lazy`.
- KMP: Koin, kotlin-inject, Metro or manual — Hilt stays Android-side. Dynamic features need plain Dagger.

Day 8 is next → Jetpack Compose fundamentals. Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700